

Name	Swamini Gawande
Section	A1
Roll no.	60
Batch	B4
Subject	Design Analysis and Algorithm

Q1. Real-World Application: Sensor Placement in IoT Networks

Question: Model a smart city IoT sensor network as a graph where edges represent communication links between sensors.

1. Formulate the sensor placement optimization as a Vertex Cover Problem.
2. Implement and compare the following algorithms for selecting minimum monitoring nodes:
 - o Greedy 2-approximation
 - o LP-relaxation
 - o Randomized heuristic
3. Use NetworkX to generate a synthetic IoT graph (500–1000 nodes, varying density).
4. Measure and compare:
 - o Number of selected sensors
 - o Execution time
 - o Coverage completeness
5. Discuss trade-offs between computational time and solution accuracy.
6. Suggest improvements (e.g., hybrid algorithms, local search, metaheuristics).

Expected Outcome:

Bridges theoretical graph algorithms to real-world optimization scenarios.

Approach:

A smart city network where many IoT sensors (like traffic lights, pollution sensors, etc.) are connected to each other.

Each connection (edge) means two sensors can communicate or monitor each other.

Greedy 2-Approximation

- Repeatedly pick a sensor connected to the most uncovered links.
- Fast and simple, but not always the smallest possible set.
- The result is at most *twice* the size of the optimal solution.

LP Relaxation (Linear Programming)

- Turns the vertex cover problem into a math optimization equation.
- Solves a “fractional” version (can pick 0–1 values as real numbers).
- Then rounds results to get an approximate integer cover.
- More accurate but slower, especially on large graphs.

Randomized Heuristic

- Randomly chooses uncovered edges and picks one endpoint each time.
- Similar to the randomized algorithm in the earlier vertex cover experiment.
- Very fast and easy to parallelize, but gives variable results.

Algorithm	Accuracy	Speed	
Greedy 2-Approx	Good	Fast	Easy to implement
LP-Relaxation	Very Good	Slow	Best accuracy, heavy computation
Randomized	Moderate	Very Fast	Useful for very large networks

Real-World Connection

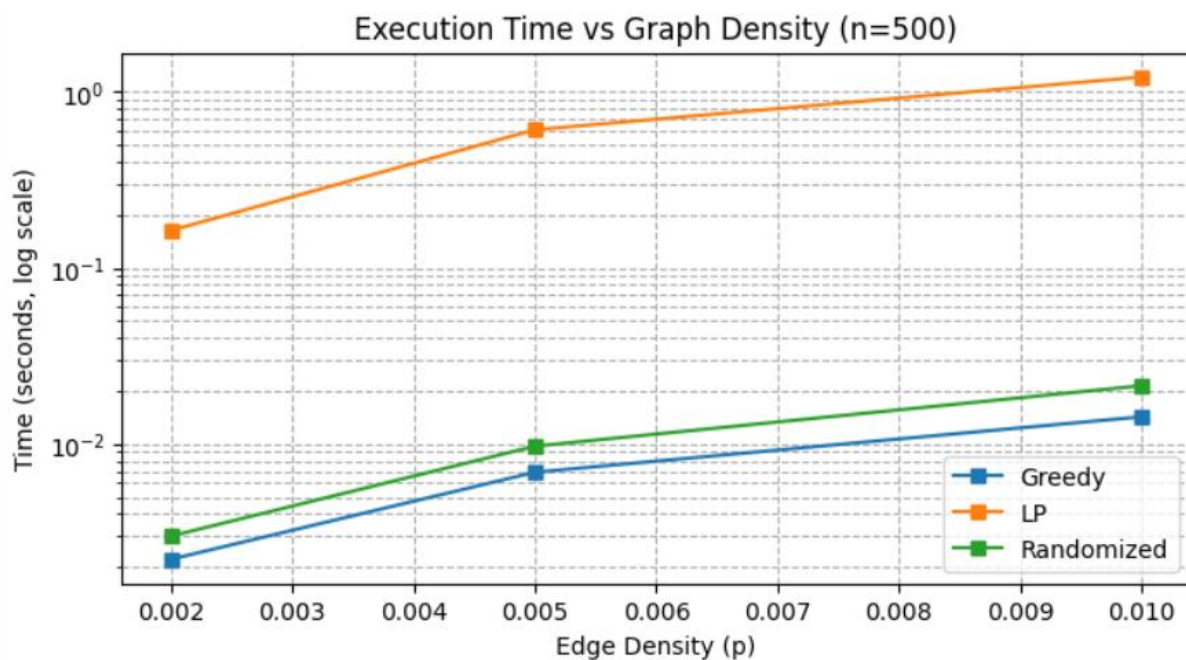
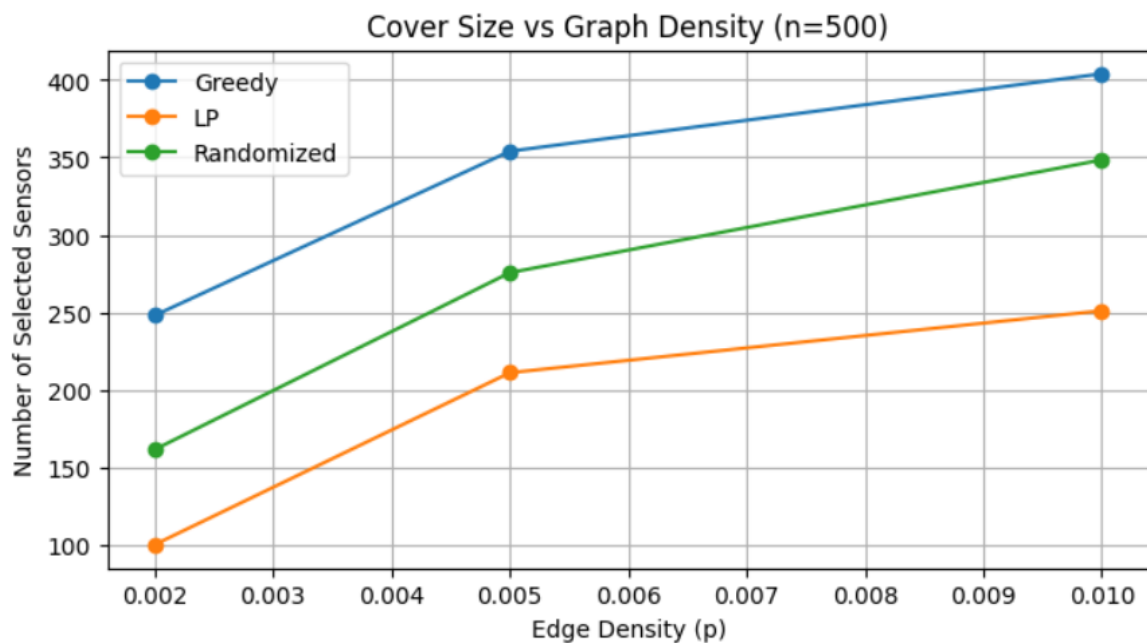
In real IoT systems:

- Each selected “sensor” might be a monitoring hub or gateway node.
- We want as few hubs as possible (to save power/cost), but still ensure every communication link is monitored.

Output:

Graphs with comparison:

Graph $n=500$, $p=0.002$ -> Done
Graph $n=500$, $p=0.005$ -> Done
Graph $n=500$, $p=0.010$ -> Done



Randomized and Probabilistic Vertex Cover Algorithms

Question 2:

In massive networks, deterministic algorithms are often infeasible. Randomized algorithms

can yield efficient approximate vertex covers.

1. Design a randomized algorithm for Vertex Cover based on the following idea:

- o Randomly select an uncovered edge (u,v) ,
- o Randomly include either u or v in the vertex cover,
- o Repeat until all edges are covered.

2. Implement the algorithm in Python or C++ and run it on multiple random graphs

(generated using NetworkX).

3. Compute and plot:

- o Average vertex cover size
- o Variance of results over 100 trials
- o Execution time vs. graph density

4. Compare its expected approximation ratio to deterministic algorithms.

5. Extend your implementation to parallel processing (e.g., using multiprocessing or

CUDA if applicable) for handling large graph instances (1000+ nodes).

6. Provide a probabilistic analysis showing expected approximation ratio ≤ 2 .

Expected Outcome:

Students explore Monte Carlo techniques, randomization effects, and performance

optimization for large-scale graphs.

Approach:

The code tests two algorithms for the Vertex Cover problem on randomly generated graphs:

1. A randomized algorithm — repeatedly picks a random uncovered edge and randomly includes one of its vertices in the cover.
2. A deterministic 2-approximation algorithm — builds a vertex cover from a maximal matching (a standard greedy method).

It generates random graphs using NetworkX, runs both algorithms many times (Monte Carlo trials), and records:

- The average vertex cover size
- The variance in results (how much the randomized outcomes differ)
- The execution time for each algorithm

Then it plots graphs showing:

1. Vertex cover size vs. graph density
2. Variance vs. graph density
3. Execution time vs. graph density

The program also computes the empirical approximation ratio, showing how close the randomized algorithm is to the deterministic one (usually ≤ 2 , as theory predicts).

It supports parallel processing (via multiprocessing) to handle large networks (1000+ nodes) efficiently.

Aggregated Results:

	p	avg_rand_size	var_rand_size	avg_det_size	var_det_size	\
0	0.001	19.4	18.933333	36.4	58.488889	
1	0.005	67.9	16.322222	110.2	36.844444	
2	0.010	102.7	24.455556	158.0	53.333333	
3	0.020	134.1	9.655556	196.0	3.555556	
4	0.050	165.9	22.988889	200.0	0.000000	

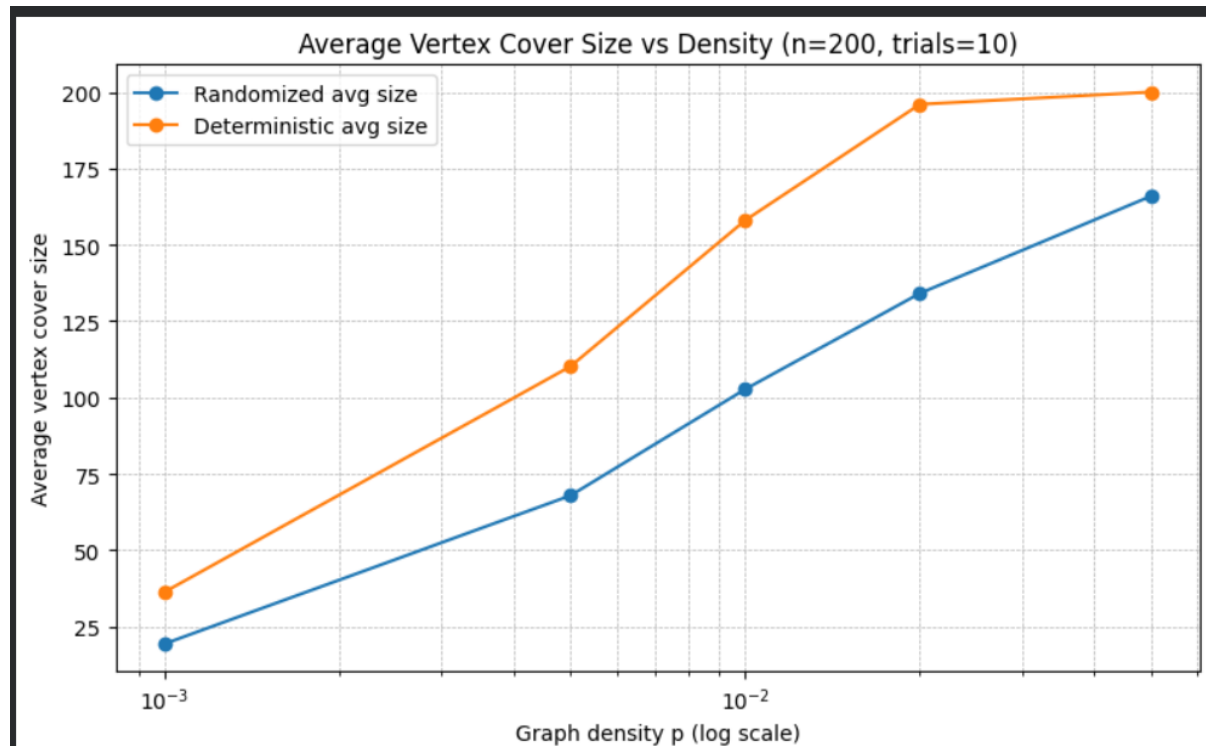
	avg_rand_time	avg_det_time
0	0.000156	0.003078
1	0.000582	0.008619
2	0.000734	0.009468
3	0.001345	0.014725
4	0.002829	0.014329

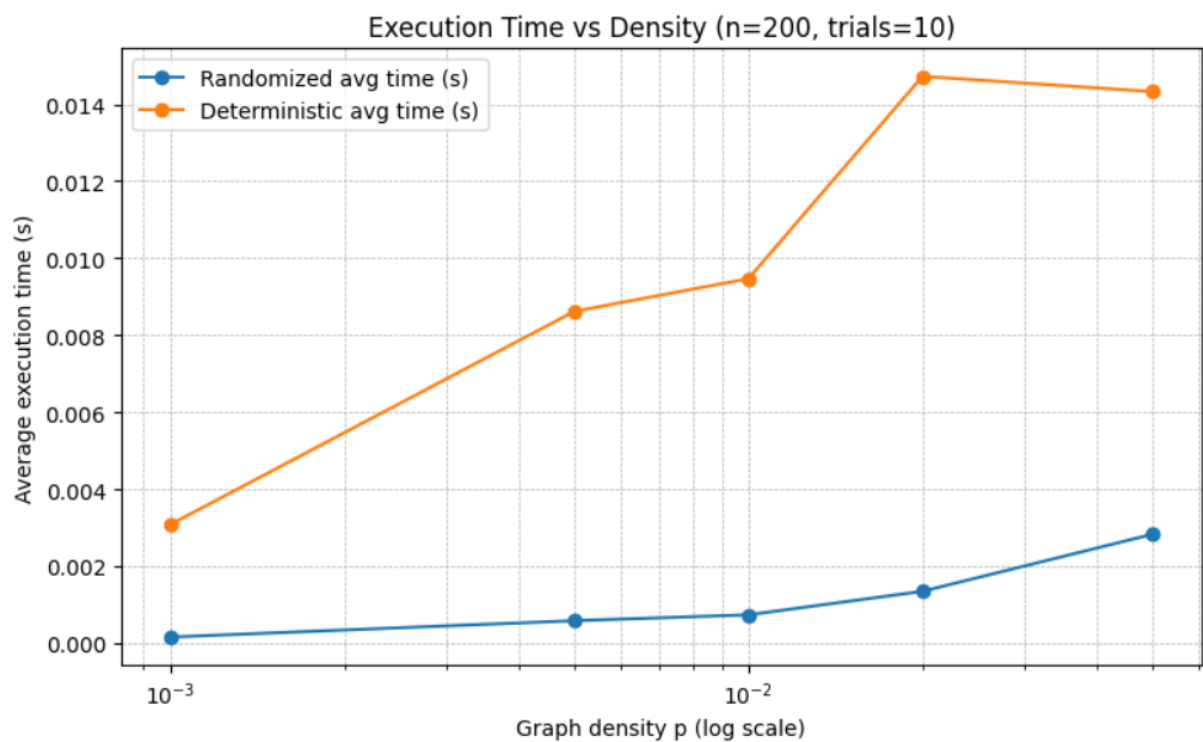
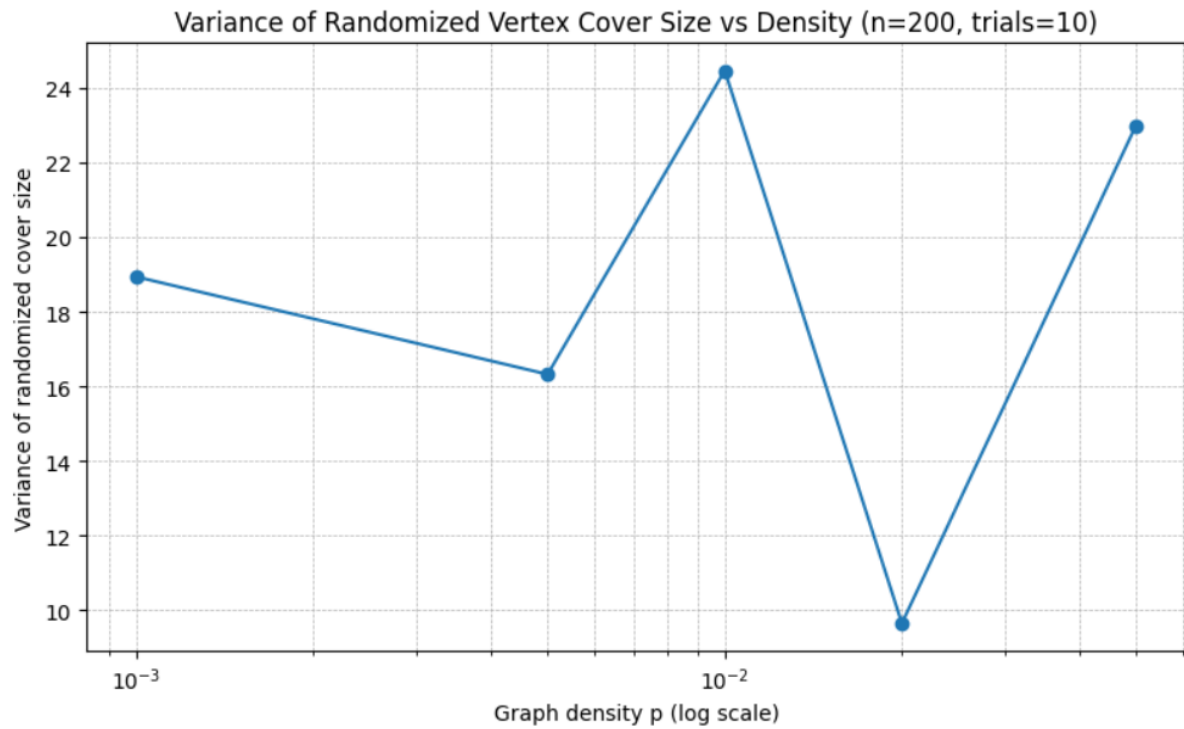
Empirical Approximation Ratio (randomized / matching size):

	p	emp_avg_rand_over_matching
0	0.001	1.063120
1	0.005	1.233804
2	0.010	1.300675
3	0.020	1.368460
4	0.050	1.659000

Output:

Graphs





3. N-Queen Variation: Blocked Cells

Problem Statement:

Modify the N-Queen problem to handle blocked cells (cells where queens cannot be placed).

For a given $N \times N$ board, some cells are marked as unavailable ("X").

Tasks:

Input: $N = 5$ and a set of blocked coordinates.

Output all possible arrangements avoiding blocked cells.

Discuss the change in solution count compared to the standard N-Queen problem.

Example Input:

$N = 5$

Blocked cells = [(0,2), (2,3), (3,0)]

Expected Output:

All valid queen placements excluding blocked cells.

Approach:

The program solves a modified N-Queen problem where some cells are blocked (marked "X") — queens cannot be placed there.

It takes user input for:

- Board size N
- List of blocked cell coordinates

Then it uses backtracking to place queens' row by row, checking:

- No two queens attack each other (same column or diagonals)
- No queen is placed on a blocked cell

Every valid arrangement is stored, and each solution is visualized using a chessboard plot (queens as crown, blocked cells as black squares).

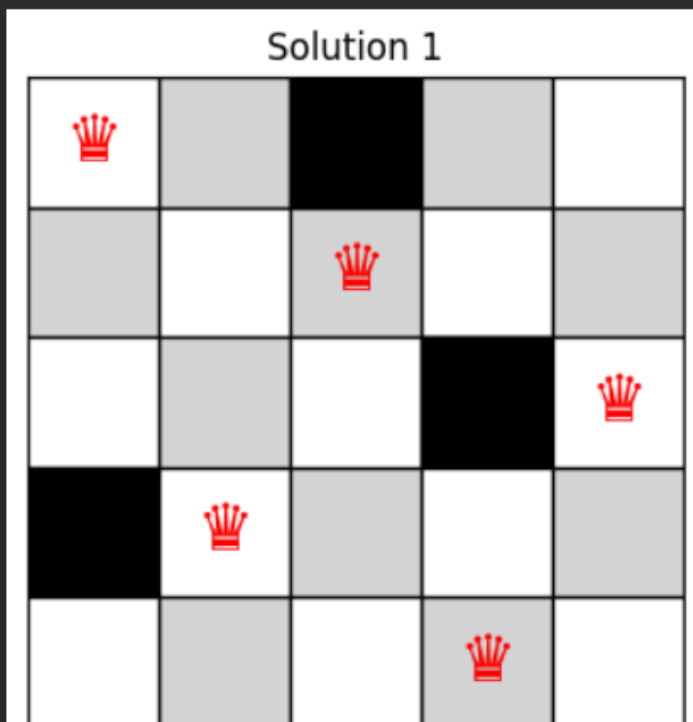
Finally, it prints the total number of valid solutions found.

Output:

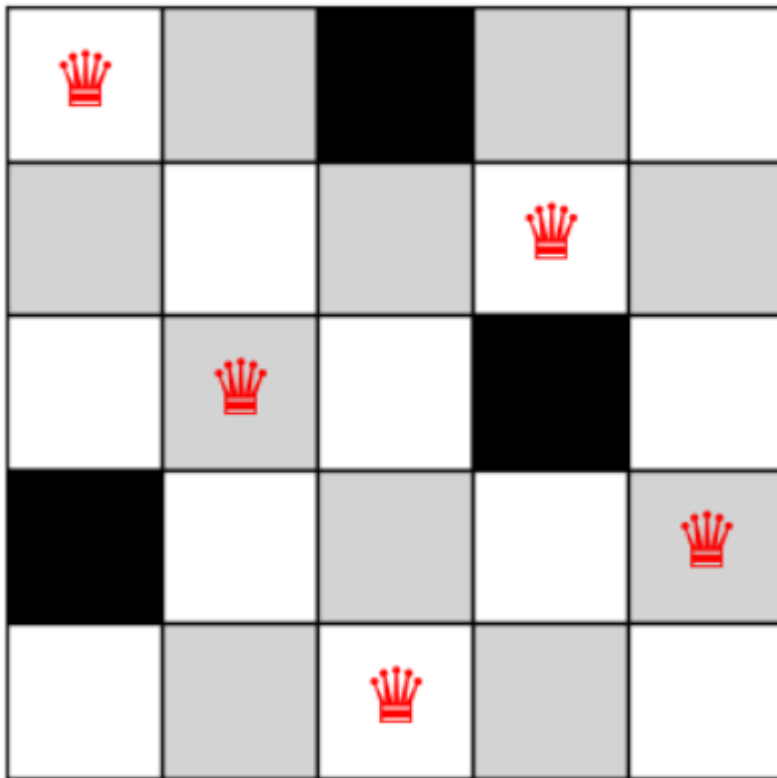
```
Enter N (board size): 5
Enter blocked cells as row,col pairs (e.g., 0,2). Type 'done' to finish:
0,2
2,3
3,0
done
```

```
Blocked cells: [(0, 2), (2, 3), (3, 0)]
```

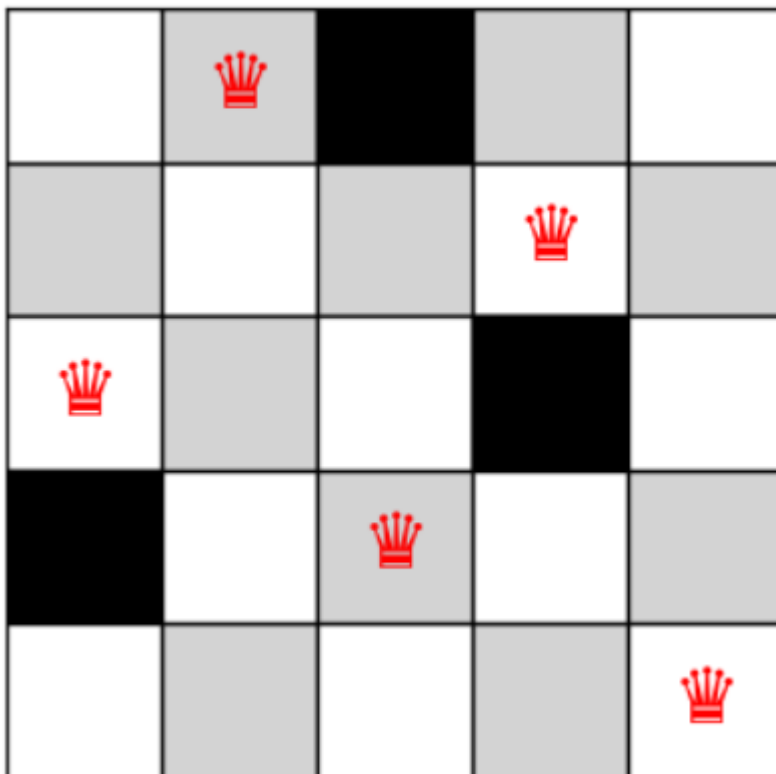
```
Total solutions found: 6
```



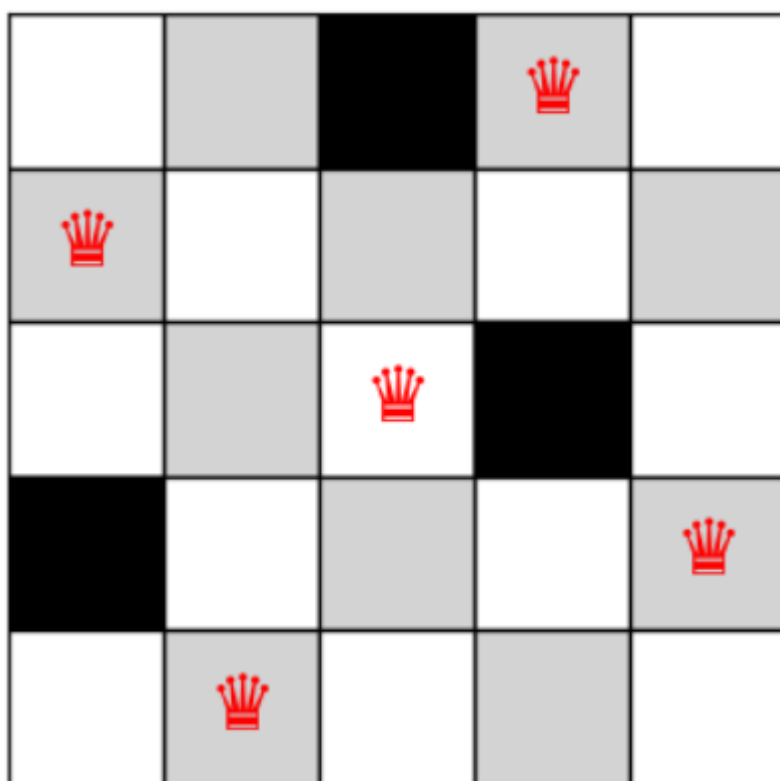
Solution 2



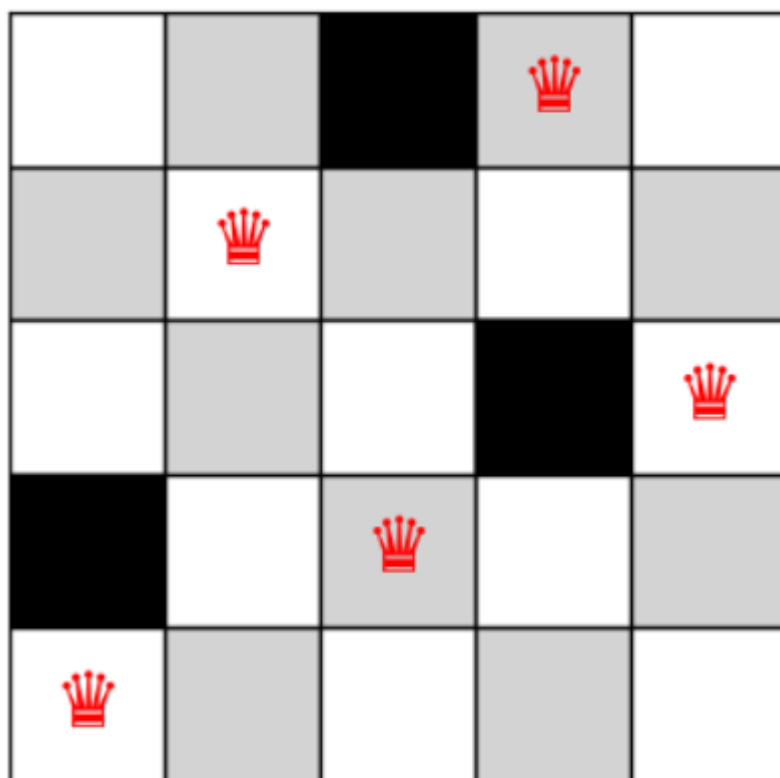
Solution 3



Solution 4



Solution 5



Solution 6

